Федеральное государственное бюджетное образовательное учреждение
высшего профессионального образования
«Казанский национальный исследовательский технологический университет»
Кафедра информатики и прикладной математики

ИНФОРМАЦИОННО-КОММУНИКАЦИОННЫЕ СИСТЕМЫ И СЕТИ

ЛАБОРАТОРНАЯ РАБОТА №7. ОРГАНИЗАЦИЯ ВЗАИМОДЕЙСТВИЯ МЕЖДУ УЗЛАМИ В ИНТЕРНЕТЕ

Методические указания к лабораторной работе

Казань
КНИТУ
2013

Цель работы: Ознакомиться с некоторыми ключевыми службами интернета: *DNS*, *WWW*, *FTP*, *e-mail*, *IM*. Получить практический навык по их конфигурированию и использованию.

Технология *клиент–сервер*

Ежедневно вы используете сетевые службы и интернет для обмена данными и выполнения различных задач обработки информации. Прием электронной почты, ввод информации в *блог* или совершение покупок через электронные магазины возможны благодаря работе серверов, клиентских программ и сетевых устройств. В большинстве наиболее популярных интернет-приложений происходят весьма сложные взаимодействия между различными серверами и клиентами.

Под термином *сервер* понимается узловая машина, на которой выполняется прикладное программное обеспечение, предоставляющее информацию или службы для других узлов, подключенных к сети. Типичным примером такой прикладной программы является *веб-сервер*. К интернету подключены миллионы серверов, предоставляющих различные услуги. Для обеспечения их надежного функционирования необходимо правильно подобрать необходимые стандарты связи и протоколы.

Для передачи запроса *веб-сайту* и просмотра *веб-страницы* пользователь обращается к устройству, на котором запущено программное обеспечение *веб-клиента*. Под термином *клиент* понимается прикладная программа, которая используется для получения доступа к информации, хранящейся на сервере. Типичным примером клиентского приложения является *веб-обозреватель*, или *браузер*.

Главной особенностью клиент–серверных систем является отправка клиентом запроса серверу. Сервер, отвечая на запрос, выполняет ту или иную функцию, например, передачу клиенту запрашиваемой информации. Типичным примером клиент–серверной системы является комплекс браузера и веб-сервера.

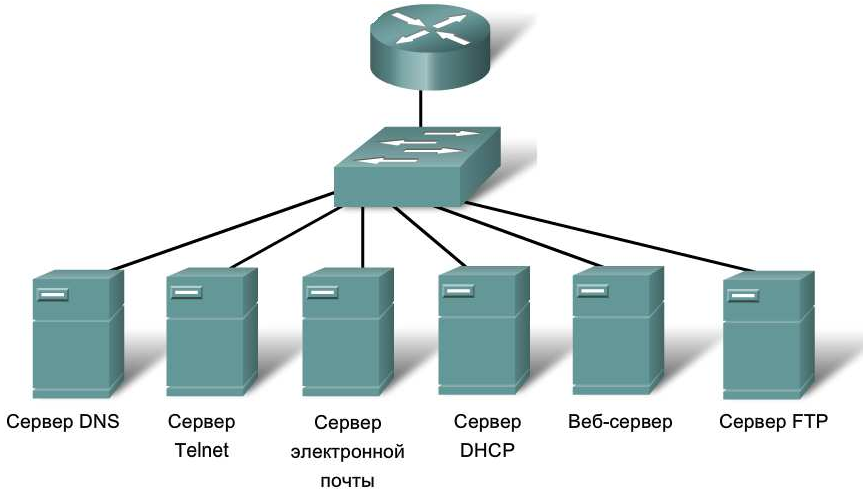

Рис. 1. Ферма серверов

В процессе обмена информацией веб-сервер и веб-клиент используют специальные протоколы и стандарты, гарантирующие прием и прочтение информации. К ним относятся следующие:

- протоколы прикладного уровня;
- транспортные протоколы;
- протоколы межсетевого взаимодействия и сетевого доступа.

Протоколы клиент–серверного взаимодействия

Одним из прикладных протоколов является *протокол передачи гипертекста* (*Hypertext Transition Protocol – HTTP*). Он управляет взаимодействием между веб-сервером и веб-клиентом. Протокол *HTTP* задает формат запросов и ответов на запросы, пересылаемых между клиентом и сервером. Для управления процессом передачи сообщений между клиентом и сервером *HTTP* обращается к другим протоколам.

Протокол управления передачей (*Transmission Control Protocol – TCP*) – это транспортный протокол, управляющий отдельными сеансами связи между веб-серверами и веб-клиентами. Он делит гипертекстовые сообщения (*HTTP*) на сегменты и отправляет их на узел назначения. Он также осуществляет управление потоками данных и подтверждает обмен пакетами между узлами.

Из протоколов межсетевого взаимодействия чаще всего применяется *межсетевой протокол* (*Internet Protocol – IP*). *IP*-протокол отвечает за прием

форматированных сегментов от *TCP*, присвоение им локальных адресов, их инкапсуляцию в пакеты для маршрутизации на узел назначения.

Из протоколов сетевого доступа в локальных сетях чаще всего применяется протокол *Ethernet*. *Протоколы сетевого доступа* выполняют две основных функции – управление каналами передачи данных и физическую передачу данных по сети.

Протоколы управления каналами передачи данных принимают пакеты от *IP*-протокола и инкапсулируют их в соответствующий формат кадров локальной сети. Эти протоколы отвечают за назначение физических адресов кадрам данных и их подготовку к передаче по сети.

Стандарты и *протоколы физической передачи данных* отвечают за представление битов в среде передачи, выбор способа передачи сигналов и их преобразование на принимающем узле. Сетевые интерфейсные платы поддерживают соответствующие протоколы среды передачи данных.

Каждая служба, доступная по сети, имеет собственные протоколы прикладного уровня, поддерживаемые программным обеспечением сервера и клиента. Помимо протоколов прикладного уровня во всех общих интернет-службах используется протокол *IP*, отвечающий за адресацию и маршрутизацию сообщений между узлами источника и назначения.

IP-протокол отвечает только за структуру, адресацию и маршрутизацию пакетов. Он не определяет способ доставки или транспортировки пакетов. Транспортные протоколы предписывают способ передачи сообщений между узлами. Наиболее популярными из транспортных протоколов являются *протокол управления передачей* (*TCP*) и *протокол пользовательских датаграмм* (*UDP*). *IP*-протокол использует эти транспортные протоколы для обеспечения связи и передачи данных между узлами.

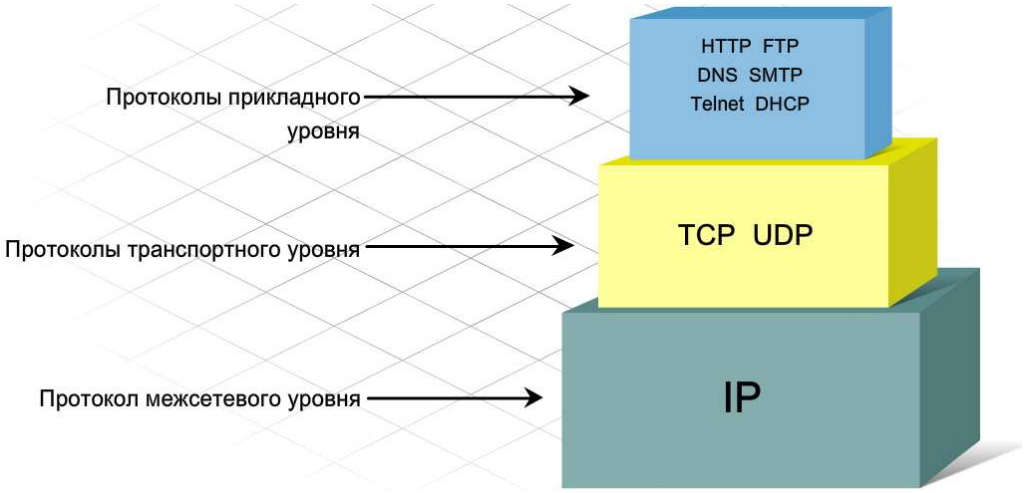

Рис. 2. Упрощенная иерархия протоколов

Если приложению требуется подтверждение доставки сообщения, оно использует протокол *TCP – надежный протокол с соединением*. Можно провести аналогию с обычной почтой, а именно – процессом отправки заказного письма в обычной почтовой системе, когда для подтверждения получения письма получатель ставит свою подпись на квитанции.

TCP разбивает сообщение на фрагменты меньшего размера, именуемые сегментами. Эти сегменты последовательно нумеруются и передаются *IP*-протоколу, который затем осуществляет сборку пакетов. *TCP* отслеживает количество сегментов, отправленных на тот или иной узел тем или иным приложением. Если отправитель не получает подтверждения в течение определенного периода времени, то *TCP* рассматривает эти сегменты как потерянные и повторяет их отправку. То есть повторно отправляется только потерянная часть сообщения, а не все сообщение целиком.

Протокол *TCP* на принимающем узле отвечает за повторную сборку сегментов сообщений и их передачу соответствующему приложению.

В некоторых случаях протокол с подтверждением доставки не требуется, так как это сильно замедляет скорость передачи данных. В таких случаях более приемлемым из транспортных протоколов является *UDP – ненадежный протокол с соединением*. Он выполняет негарантированную доставку данных и не запрашивает подтверждения от получателя. Это аналогично отправке письма

обычной почтой без уведомления о доставке. Доставка письма не гарантируется, но шансы доставки достаточно высоки.

Протокол *UDP* более предпочтителен для передачи потокового аудио, видео и голосовой связи на основе протокола *IP* (*VoIP*). Потеря некоторых пакетов не нанесет непоправимый ущерб информации, и при этом подтверждение доставки лишь замедлит процесс передачи данных. Примером использования протокола *UDP* является интернет-радио. Если какое-либо сообщение затерялось в пути доставки по сети, оно не будет отправляться повторно. Пропадание нескольких пакетов будет восприниматься слушателем как кратковременное пропадание звука. Если для этого использовать протокол *TCP*, предусматривающий повторную доставку потерянных пакетов, то процесс передачи данных приостановится для приема потерянных пакетов, что психологически заметно ухудшит качество воспроизведения.

Порты *TCP/IP*

При доставке сообщения по протоколу *TCP* или *UDP* запрашиваемые протоколы и службы распознаются по номеру порта. *Порт* – это числовой идентификатор внутри каждого сегмента, используемый для учета отдельных сеансов связи запросов услуг узлами назначения. В каждом сообщении, отправляемом с узла, указываются *порт источника* и *порт назначения*.

Клиент указывает номер порта назначения в сегменте, чтобы сообщить серверу, какая запрашивается служба. Например, порт **80** обычно обозначает протокол *HTTP* или веб-службы. Если клиент в описании для порта назначения указывает порт **80**, то сервер, принимающий это сообщение, уже «знает», что выполнен запрос веб-службы. Сервер может предоставлять несколько служб одновременно. Например, сервер может предоставлять веб-службы через порт **80** и одновременно услуги обмена файлами по протоколу *FTP* через порт **21**.

Номер порта источника генерируется случайно устройством-отправителем для идентификации сеанса связи между двумя устройствами. Это позволяет устанавливать одновременно несколько сеансов связи. Другими словами, запросы

службы *HTTP* с веб-сервера могут направляться несколькими устройствами одновременно. Отдельные сеансы связи отслеживаются по номерам портов источника.

Номера порта источника и порта назначения записываются в сегмент. Затем эти сегменты инкапсулируются в пакете *IP*. В пакете *IP* записывается *IP*-адрес источника и назначения. Комбинация *IP*-адресов источника и назначения, а также номеров портов источника и назначения называется *сокетом*. Сокет используется для определения сервера и служб, запрашиваемых клиентом. Ежедневно тысячи узлов взаимодействуют с тысячами различных серверов. Их взаимодействия определяются с помощью сокетов.

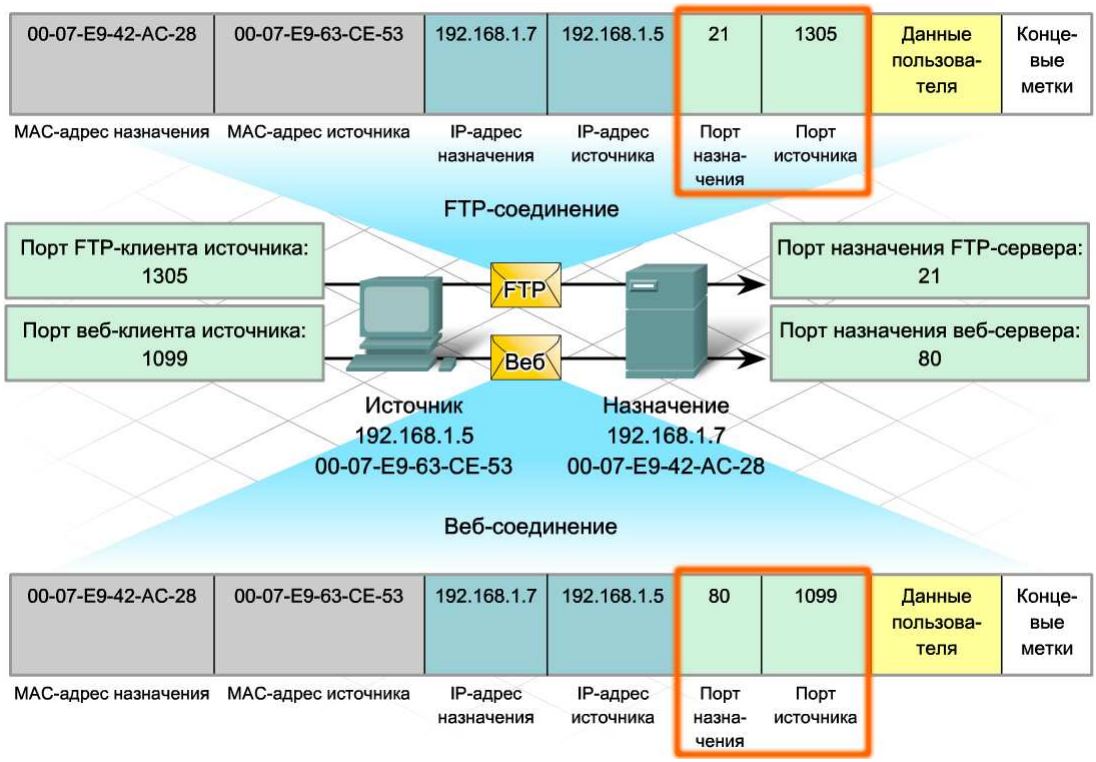

Рис. 3. Взаимодействие адресов и портов

Порты подразделяются на три категории и нумеруются в диапазоне от **1** до **65535**. За присвоение портов и управление ими отвечает *Internet Corporation for Assigned Names and Numbers* (*ICANN*).

1. *Известные порты*, т.е. порты назначения, ассоциированные с общими сетевыми приложениями, имеют диапазон номеров от **1** до **1023**.

2. *Зарегистрированные порты* с номерами от **1024** до **49151** могут использоваться в качестве портов источника и назначения. Эти номера использу-

ются организациями для регистрации отдельных приложений, например, программ мгновенного обмена сообщениями.

3. *Частные порты* с номерами от **49152** до **65535** часто используются в качестве портов источника. Эти порты могут использоваться любым приложением.

Например, некоторых из известных портов:

- *FTP*: 21 – для команд, 20 – для данных;
- *SSH*: 22 – удаленный доступ с шифрованием;
- *telnet*: 23 – удаленный доступ;
- *SMTP*: 25, 465, 587 – почтовая служба;
- *DNS*: 53 (*UDP*) – служба доменных имен;
- *DHCP*: 67, 68/*UDP* – служба динамического распределения адресов;
- *TFTP*: 69/*UDP* – туннельный *FTP*;
- *HTTP*: 80, 8080 – гипертекстовая передача;
- *POP3*: 110 – почтовая служба;
- *NTP*: 123 (*UDP*) – служба времени;
- *IMAP*: 143 – комплексная почтовая служба;
- *SNMP*: 161 – протокол управления сетью;
- *HTTPS*: 443 – гипертекстовая передача с шифрованием;
- *MySQL*: 3306 – доступ к базам данных *MySQL*;
- *RDP*: 3389 – удаленный доступ к рабочему столу;
- *OSCAR* (*ICQ*): 5190 – пиринговый обмен короткими сообщениями для клиента службы *ICQ*;
- *XMPP* (*Jabber*): 5222/5223 – клиент-сервер, 5269 – сервер-сервер для службы обмена короткими сообщениями *Jabber*;
- *BitTorrent*: 6969, 6881-6889 – пиринговый обмен файлами для службы *BitTorrent*.

Наиболее полный список используемых сегодня портов с описаниями на русском языке можно найти в интернете по адресу **http://ru.wikipedia.org/wiki/Список_портов_TCP_и_UDP**.

Служба доменных имен

Каждый узел в сети имеет собственный уникальный *IP*-адрес, порой даже не один. В общем случае невозможно запомнить все *IP*-адреса всех серверов и сайтов, предоставляющих различные услуги в интернете. Вместо этого был разработан более удобный способ поиска серверов – сопоставление *IP*-адреса с некоторым именем.

Служба доменных имен (*DNS*) позволяет использовать имя узла для запроса *IP*-адреса отдельного сервера. Регистрация и организация имен в этой системе выполняется по специальным высокоуровневым группам – *доменам*. К числу наиболее популярных высокоуровневых доменов сети Интернет относятся **.com**, **.edu** и **.net**.

В *DNS*-сервере записана специальная таблица, ассоциирующая имена узлов в домене с соответствующими *IP*-адресами. Если клиент знает имя веб-сервера, и требуется найти *IP*-адрес, он направляет запрос на *DNS*-сервер через порт 53. Клиент использует *IP*-адрес *DNS*-сервера, прописанный в настройках узла.

При получении запроса *DNS*-сервер выясняет по своей таблице, имеется ли соответствие между запрашиваемым *IP*-адресом и веб-сервером. Если на *DNS*-сервере отсутствует запись о запрашиваемом имени, он опрашивает другой *DNS*-сервер в пределах своего домена. После распознавания *IP*-адреса *DNS*-сервер отправляет результат клиенту. Если *DNS*-серверу не удалось определить *IP*-адрес, клиент не сможет установить связь с этим веб-сервером и получит сообщение об истечении времени ожидания.

Процесс определения *IP*-адреса по *DNS*-протоколу из клиентского программного обеспечения обычно достаточно прост и прозрачен для пользователя. Программа самостоятельно или посредством стандартных процедур опера-

ционной системы определяет соответствующий адрес, не беспокоя пользователя лишними вопросами.

Служба *WWW*

После получения *IP*-адреса веб-сервера веб-клиентом обозреватель на стороне клиента инициирует запрос веб-служб по этому *IP*-адресу и направит его через порт **80**. Этот запрос отправляется на сервер по протоколу передачи гипертекстовых файлов (*HTTP*).

После приема запроса на порт **80** сервер отвечает на клиентский запрос и отправляет клиенту запрашиваемую веб-страницу. Информационное наполнение веб-страницы кодируется с помощью специализированных языков разметки. *HTML* (язык гипертекстовой разметки) является наиболее популярным, но также широко используются такие языки, как *XML* и *XHTML*.

Протокол *HTTP* не является безопасным; информация может быть легко перехвачена при передаче по сети, например, при помощи *сетевого сниффера*. Для обеспечения безопасности данных возможны комбинации протокола *HTTP* с безопасными транспортными протоколами. Запросы по безопасному протоколу *HTTP* направляются на порт **443**. В таких запросах используется префикс **`https:`**, а не **`http:`**.

Сочетание протоколов *HTTP* и *HTML* обеспечивает эффективное взаимодействие серверов и клиентов от различных поставщиков. Служба, основанная на нем, называется *WWW – World Wide Web*. Часто *WWW* ошибочно ассоциируют со всем интернетом. Это, конечно же, не так. Служба *WWW* действительно вобрала в себя многие протоколы, помимо *HTTP* и *HTML*, и способна реализовать множество подходов к обработке информации. Тем не менее, существует ряд других, не менее важных и полезных, служб.

Служба *FTP*

Для обмена файлами в интернете часто используется служба *WWW*. Однако она довольно плохо приспособлена для двусторонней передачи файлов, осо-

бенно больших. В этом случае часто используют другую службу, основанную на протоколе *FTP*.

Протокол передачи файлов (*FTP*) обеспечивает простой обмен файлами между компьютерами. Узел, на котором выполняется программное обеспечение *FTP*-клиента, обращается к *FTP*-серверу, выполняющему различные функции управления файлами, в том числе выгрузку и загрузку файлов.

FTP-сервер обеспечивает обмен файлами между клиентом и устройствами. С помощью *FTP*-сервера клиент может также управлять файлами удаленно, например, с помощью команд удаления или переименования файлов. Для этого служба *FTP* использует два порта обмена данными между клиентом и сервером.

Запросы на запуск сеанса *FTP* передаются на сервер через порт назначения **21**. После открытия сеанса связи сервер переключится на порт **20**, через который выполняется передача файлов данных.

Программное обеспечение *FTP*-клиента обычно имеется в составе операционной системы компьютера, а также в режиме ограниченной функциональности встроено в большинство веб-обозревателей. В автономных *FTP*-клиентах предусмотрен целый ряд функций управления файлами на сервере через графический интерфейс.

Электронная почта

Одна из старейших и популярнейших служб в интернете – *электронная почта* (*electronic mail*, *e-mail*). Первое электронное письмо было отправлено еще в 1965 г. из Массачусетского технологического института с компьютера *IBM 7090/7094*. Появление и развитие *e-mail* оказало и до сих пор оказывает огромное влияние на развитие интернета.

Электронная почта – одно из наиболее популярных клиент-серверных приложений в интернете. Серверное программное обеспечение, работающее на серверах электронной почты, обеспечивает взаимодействие с другими серверами электронной почты в сети.

Каждый сервер электронной почты отвечает за прием и хранение почты пользователей в индивидуальных почтовых ящиках, созданных на сервере электронной почты. Чтобы обратиться к своему почтовому ящику на сервере электронной почты и прочитать сообщения, пользователь должен использовать программу клиента электронной почты.

Серверы электронной почты также используются для отправки почты, адресованной на локальные почтовые ящики или на почтовые ящики, расположенные на других серверах электронной почты.

Сегодня принят следующий формат почтового адреса:

`user@company.domain`

Для обработки почты применяются различные протоколы прикладного уровня, в том числе *SMTP*, *POP3*, *IMAP4*.

Простой протокол передачи почты (*SMTP*) используется программой клиента электронной почты для отправки сообщений на локальный сервер электронной почты. Далее локальный сервер определяет, кому адресовано сообщение – локальному почтовому ящику или почтовому ящику на другом сервере.

Протокол *SMTP* применяется также при взаимодействии серверов, например, если требуется отправка сообщения на другие серверы. *SMTP*-запросы направляются на порт **`25`**.

Post Office Protocol (*POP3*) – протокол, выполняющий на сервере функции приема и хранения сообщений для пользователей. После того, как установлено соединение между клиентом и сервером электронной почты, сообщения будут загружены на компьютер клиента. По умолчанию сообщения не сохраняются на сервере после их прочтения клиентом. Клиенты обращаются к серверам *POP3* через порт **`110`**.

Рис. 4. *POP3/SMTP*: клиент и сервер электронной почты

Протокол *IMAP4* является альтернативой протоколу *SMTP*. *IMAP*-сервер также принимает и сохраняет сообщения, адресованные его пользователям. Тем не менее, сообщения могут храниться в почтовых ящиках пользователей, если они не будут явно удалены самими пользователями. В самой последней версии протокола *IMAP* – *IMAP4* запросы от клиентов прослушиваются через порт **143**. Важной функцией *IMAP* является возможность выборочного частичного получения сообщения для оценки его важности.

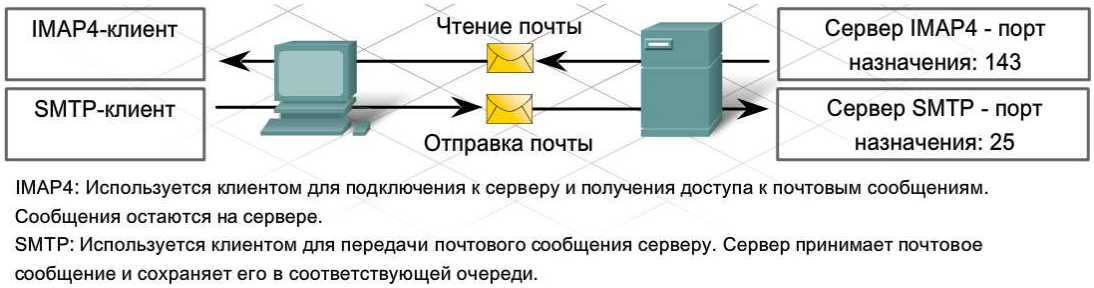

Рис. 5. *IMAP4/SMTP*: клиент и сервер электронной почты

После установления соединения между клиентом и сервером электронной почты начинается загрузка и просмотр сообщений. Большинство клиентов электронной почты настроены либо на POP3, либо на IMAP4 в зависимости от типа сервера электронной почты, на котором зарегистрирован почтовый ящик. Клиенты электронной почты должны отправлять электронную почту на сервер по протоколу SMTP.

Серверы электронной почты настраиваются на обработку входящей и исходящей почты.

Обычно при настройке клиента электронной почты требуется сконфигурировать следующие параметры:

- имя сервера POP3 или IMAP4;
- имя сервера SMTP;
- имя пользователя;
- пароль пользователя;
- фильтры нежелательной почты.

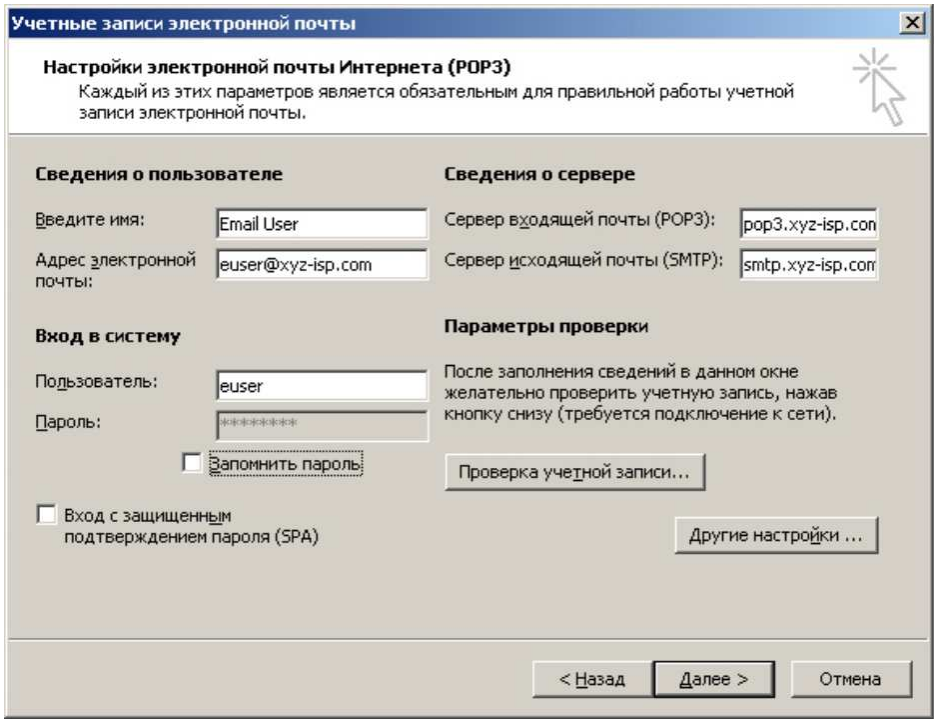

Рис. 6. Окно конфигурации учетной записи в *e-mail* клиенте *Microsoft Outlook*

Мгновенный обмен сообщениями

Мгновенный обмен сообщениями (*Instant Messaging, IM*) – один из наиболее популярных сервисов обмена информацией. Программное обеспечение мгновенного обмена сообщениями, выполняемое на локальных компьютерах, обеспечивает взаимодействие пользователей в окнах передачи сообщений в интернете в реальном времени. На рынке сегодня предлагается множество различных программ мгновенного обмена сообщениями. В каждой такой службе могут использоваться специальные протоколы и порты, поэтому на двух разных узлах должно быть установлено совместимое программное обеспечение.

Для работы клиентского приложения достаточно ввести имя пользователя и пароль. Эта операция необходима для проверки подлинности клиента на вхо-

де в сеть мгновенного обмена сообщениями. После выполнения входа на сервер клиенты могут отправлять сообщения другим клиентам в реальном времени. Помимо текстовых сообщений *IM*-клиенты зачастую поддерживают передачу видео, музыкальных файлов и файлов голосовой связи, а также позволяют осуществлять голосовые вызовы через интернет. В клиентских приложениях *интернет-телефонии* реализована технология обмена данными между равноправными уровнями (*peer-to-peer technology*, *P2P*). В *IP*-телефонии применяется технология *VoIP*, в которой *IP*-пакеты используются для передачи оцифрованной речи в виде данных.

Рис. 7. Интернет-телефония

DNS, *WWW*, *e-mail*, *FTP*, *IM* и *VoIP* – это лишь некоторые из многочисленных служб, доступных в клиент-серверных системах через интернет. Эти службы могут быть доступны с одного или нескольких серверов.

В любом случае серверу необходимо знать, какая услуга запрашивается клиентом. Это позволяет идентифицировать запросы от клиентов, так как запрос затем направляется на определенный порт назначения. Клиентскому приложению присваивается предварительно заданный порт назначения, зарегистрированный в интернете для каждой службы. Все данные, поступающие на такой порт, система автоматически передает сопоставленному ему приложению.

Взаимодействие протоколов

Для успешного взаимодействия между узлами необходимо эффективное взаимодействие целого ряда протоколов. Эти протоколы реализованы на уровне оборудования и программного обеспечения каждого сетевого устройства.

Взаимодействие между протоколами можно представить в виде *стека протоколов*. Протоколы в стеке представляют собой многоуровневую иерархию, в которой протокол верхнего уровня зависит от служб протоколов на более низких уровнях.

На рис. 8 показан стек протоколов с набором первичных протоколов, необходимых для запуска веб-сервера в сети *Ethernet*. Нижние уровни стека отвечают за перемещение данных по сети и предоставление служб верхним уровням. Верхние уровни в большей степени отвечают за наполнение пересылаемых сообщений и пользовательский интерфейс.

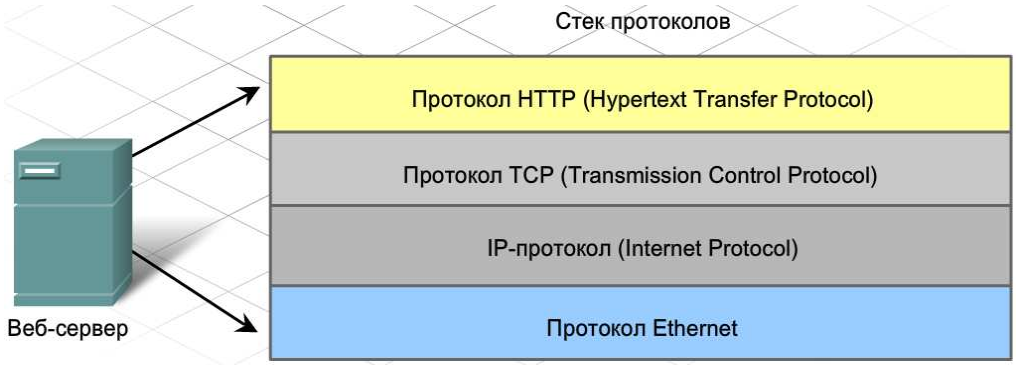

Рис. 8. Стек протоколов

Для описания взаимодействий между различными протоколами обычно применяется *многоуровневая модель*. Она описывает взаимодействие протоколов внутри каждого уровня, а также взаимодействие с верхними и нижними уровнями.

Многоуровневая модель имеет ряд преимуществ:

- она упрощает разработку протоколов, так как протоколы работают с определенным форматом данных и предоставляют интерфейс к верхним и нижним уровням;
- заставляет поставщиков конкурирующих продуктов создавать унифицированные решения;
- исключает возможности изменения технологий или функций одного уровня без учета последствий для верхних и нижних уровней;
- предоставляет общий язык для описания функций сетевого взаимодействия.

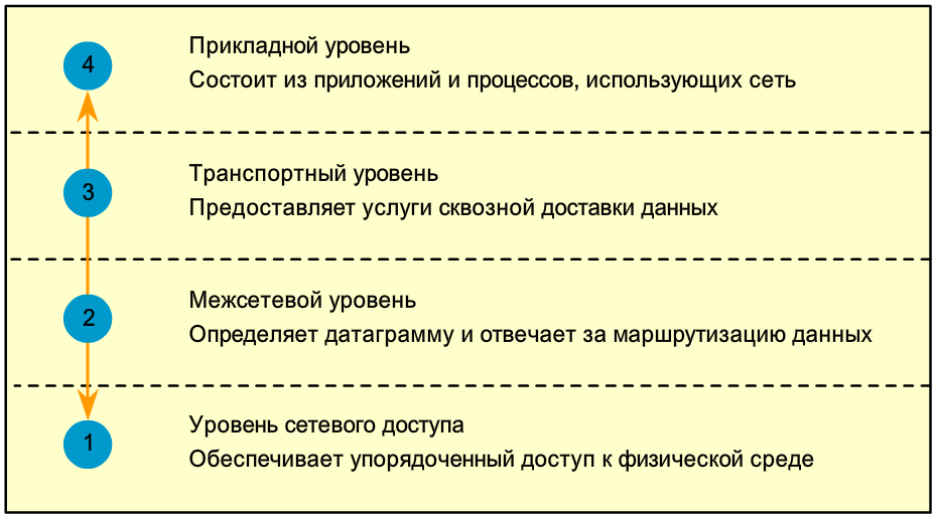

Рис. 9. Многоуровневая модель *TCP/IP*

При отправке сообщения по сети стек протоколов на узле работает от верхнего уровня к нижнему. Например, обозреватель на стороне клиента направляет веб-серверу запрос на веб-страницу через порт назначения **80**. Сервер принимает запрос, обрабатывает и отправляет веб-страницу клиенту.

При отправке веб-страницы по стеку протоколов веб-сервера файл, содержащий страницу (данные пользователя), разбивается на *TCP*-сегменты. Каждому сегменту *TCP* присваивается заголовок, в котором указываются номера портов источника и назначения, соответствующие используемой службе.

Таким образом, сегмент *TCP* инкапсулирует (скрывает) протокол *HTTP* и пользовательские данные веб-страницы в формате *HTML* и передает их на следующий уровень протоколов, то есть в *IP*. Здесь сегмент *TCP* инкапсулируется в пакете *IP*, и к нему добавляется заголовок *IP*. В заголовке *IP* указываются *IP*-адреса источника и назначения.

Далее этот пакет *IP* передается протоколу *Ethernet*, где он, в свою очередь, инкапсулируется в заголовок кадра и в концевую метку. В каждом заголовке кадра *Ethernet* указываются *MAC*-адреса источника и назначения. В концевой метке указывается информация для проверки ошибок. Наконец, биты кодируются в среду передачи по *Ethernet* (медный или оптоволоконный кабель или беспроводная среда) интерфейсной платой сервера.

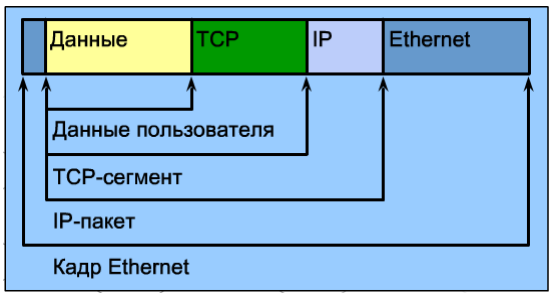

Рис. 10. Термины, описывающие инкапсуляцию протоколов

При приеме сообщений из сети стек протоколов на узле работает с нижнего уровня к верхнему. Выполняется декодирование битов интерфейсной платой на стороне клиента, а *MAC*-адрес назначения определяется клиентом как свой собственный.

Далее кадр передается по стеку протоколов, заголовок *Ethernet* (*MAC*-адреса источника и назначения) и концевая метка удаляются (*деинкапсулируются*). Остальной пакет *IP* и содержимое передаются на уровень *IP*.

На уровне *IP* заголовок *IP* (*IP*-адреса источника и назначения) удаляется, и содержимое передается на уровень *TCP*.

На уровне *TCP* заголовок *TCP* (порты источника и назначения) удаляется, а содержимое пользовательской веб-страницы передается в приложение обозревателя по протоколу *HTTP*. После приема сегментов *TCP* выполняется их повторная сборка и формируется исходный вид веб-страницы.

В отличие от модели *TCP/IP*, *модель взаимодействия открытых систем OSI* не описывает взаимодействий между отдельными протоколами. Она была создана в качестве базовой архитектуры, которую разработчики использовали для создания протоколов сетевого взаимодействия. И хотя все семь уровней модели взаимодействия открытых систем в точности реализованы в очень немногих стеках протоколов, на сегодня она считается эталонной моделью межкомпьютерных взаимодействий.

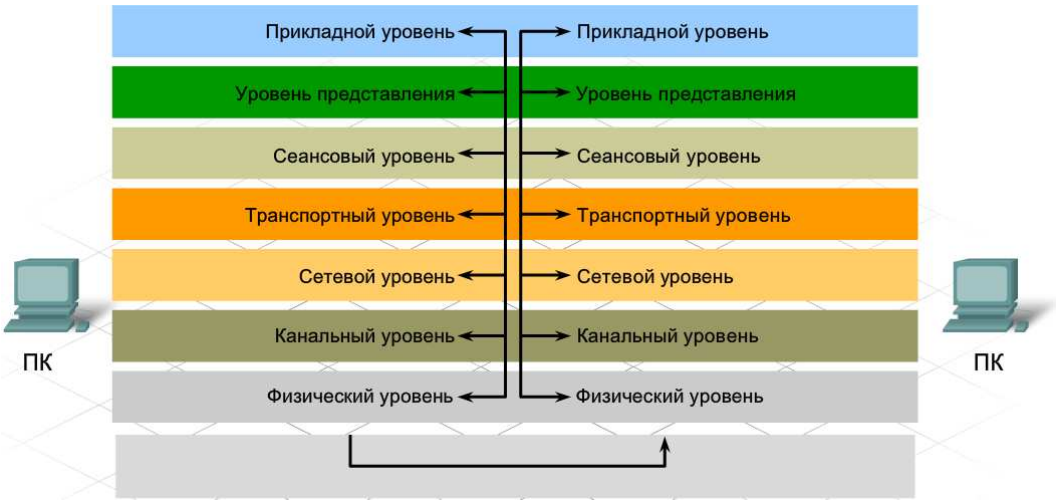

Рис. 11. Многоуровневая модель *OSI*

Семь уровней модели *OSI* можно разделить на две группы: верхние и нижние уровни.

Группа	#	Название уровня	Общие протоколы и технологии	Распространенные сетевые компоненты данного уровня
Верхние уровни	7	Приложение	DNS, NFS, DHCP, SNMP, FTP, TFTP, SMTP, POP3, IMAP, HTTP, Telnet	Приложения для контроля состояния сети, электронная почта, веб-обозреватели, передача файлов, разрешение имен
	6	Представления	SSL, оболочки и редиректоры, MIME	
	5	Сеансовый уровень	NetBIOS, интерфейсы приложений, удаленные вызовы процедуры	
Нижние уровни	4	Транспортный уровень	TCP и UDP	Механизмы потокового видео и голосовой связи, списки фильтров межсетевых экранов
	3	Сеть	IPv4, IPv6, IP NAT	IP-адресация, маршрутизация
	2	Канальный уровень	Семейство протоколов Ethernet, WLAN, Wi-Fi, ATM, PPP	Сетевые интерфейсные платы и накопители, сетевая коммутация, подключение к глобальной сети
	1	Физический уровень	Передача электрических сигналов, формы световых волн, формы радиоволн	Физическая среда передачи данных (медная витая пара, оптоволоконный кабель, беспроводные передатчики), концентраторы и ретрансляторы

Рис. 12. Верхние и нижние уровни модели OSI

Верхними уровнями часто называют все, что находится выше уровня передачи модели *OSI*. Верхние уровни относятся к работе приложений и обычно реализуются только в программном обеспечении. Наивысший уровень, называемый прикладным уровнем, находится ближе всего к конечному пользователю.

Термин «нижний уровень» иногда используется применительно к любому уровню, лежащему ниже сеансового уровня. Совместное функционирование

нижних уровней обеспечивает передачу данных. Физический уровень и уровень канала данных реализуются в аппаратном и программном обеспечении. Физический уровень ближе всего к физической среде передачи, или сетевым кабелям. Физический слой фактически помещает информацию в среду.

Конечные станции, например, клиенты и серверы, обычно работают на всех семи уровнях. Сетевые устройства работают только на нижних слоях. Концентраторы – это уровень 1, коммутаторы – уровни 1 и 2, маршрутизаторы – уровни 1, 2 и 3 и межсетевые экраны – уровни 1, 2, 3 и 4.

В модели *OSI* представлены все функции или задачи, ассоциированные с межсетевыми взаимодействиями, а не только с определенными протоколами *TCP/IP*. При этом модель *OSI* организует задачи в семь специфических групп.

Суть стека протоколов заключается в разделении и организации наиболее значимых функций. Разделение функций обеспечивает независимое функционирование каждого уровня в стеке. Например, доступ на веб-сайт возможен как с портативного компьютера, подключенного к домашнему модему, так и с портативного компьютера с помощью беспроводного или мобильного телефона с поддержкой функций беспроводного доступа. Нижние уровни не влияют и не должны влиять на эффективность работы прикладного уровня.

Точно так же нижние уровни не зависят от других уровней. Например, на скорость соединения с интернетом не влияет одновременный запуск нескольких приложений, таких, например, как электронная почта, просмотр веб-страниц, мгновенный обмен сообщениями и загрузка музыкальных файлов.

В графическом интерфейсе Packet Tracer можно просматривать эмулируемые данные, передаваемые между двумя узлами. В нем используются протокольные блоки данных (*PDU*) для представления кадров трафика сети и отображения информации о стеках протоколов на соответствующих уровнях модели *OSI*.

Как видно на рис. 13, запрос от веб-клиента принимается платой *Ethernet NIC* на веб-сервере. В модели взаимодействия открытых систем представлены четыре уровня с 1 по 4:

- уровень 1 (физический): порт *Fast Ethernet*;
- уровень 2 (канальный): *MAC*-адреса *Ethernet*;
- уровень 3 (сетевой): *IP*-адреса;
- уровень 4 (транспортный): номера портов *TCP*.

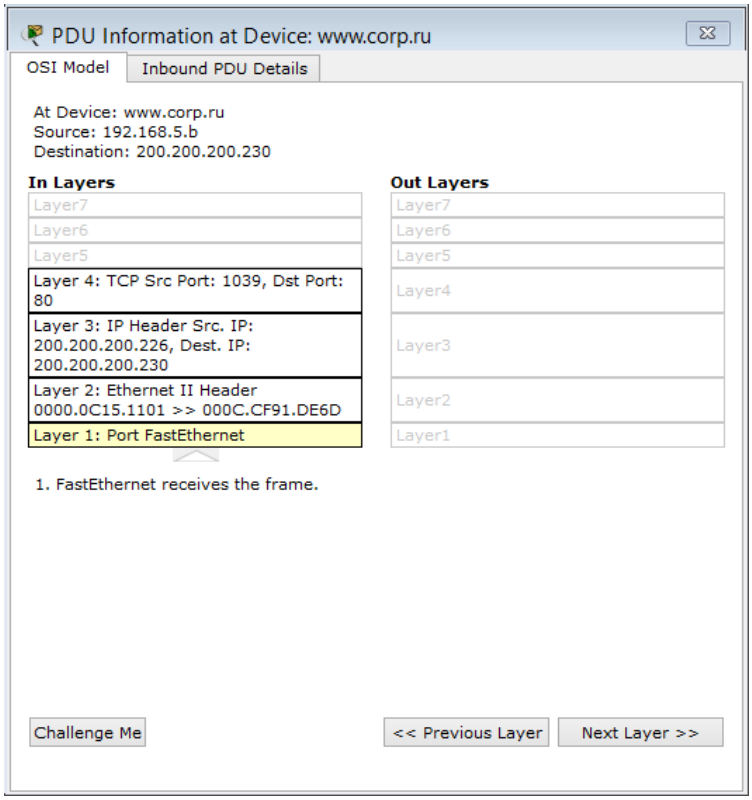

Рис. 13. Анализ уровней в HTTP-запросе

Практическая часть

1. Откройте топологию корпоративной сети (интранета), разработанную Вами при выполнении предыдущей лабораторной работы.

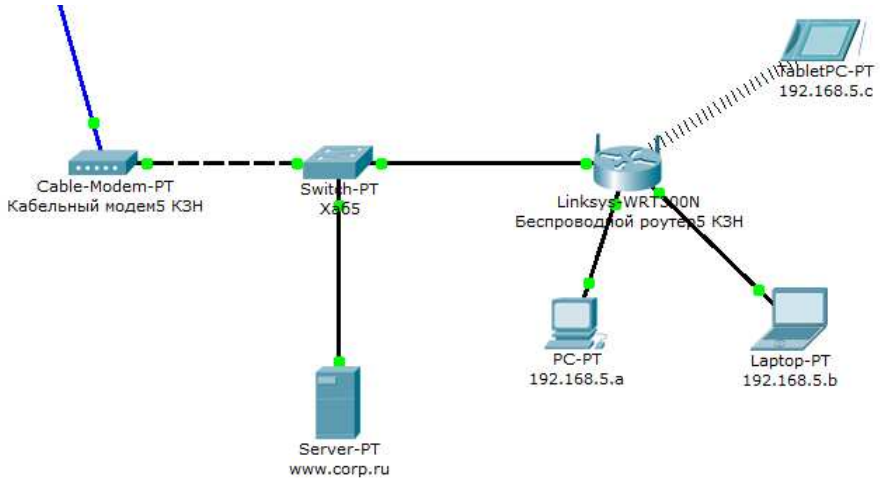

Рис. 14. Сегмент «Филиал в Казани»

2. В сегменте «Филиал в Казани» настройте *DNS*-сервер на хостинг-сервере **`www.corp.ru`** следующим образом:

- убедитесь, что *DNS*-сервер включен;
- добавьте запись типа *A* для названия сайта **`www.corp.ru`** по *IP*-адресу **`200.200.200.230`**.

3. Проверьте, получается ли с узла **`192.168.5.a`** отправить пинг на сайт **`www.corp.ru`**. Проанализируйте поведение сети.

4. Измените настройку *DNS беспроводного роутера5*, а именно, укажите в качестве первого статического *DNS*-сервера адрес **`200.200.200.230`**. Переинициализируйте сетевой адаптер узла **`192.168.5.a`** и снова отправьте с него пинг на сайт **`www.corp.ru`**. Проанализируйте поведение сети.

5. Запустите браузер на компьютерах **`192.168.5.a`**, **`192.168.1.2`** **`192.168.3.2`**, **`192.168.2.4`** и откройте во всех них сайт по адресу **`www.corp.ru`**. Проанализируйте результат.

6. Сконфигурируйте *FTP*-сервер на хостинг-сервере **`www.corp.ru`**. Добавьте пользователя **`user`** с паролем **`password`** и присвойте ему полные права доступа к серверу.

7. Подключитесь из командной строки узла **`192.168.5.a`** к службе FTP сайта **`www.corp.ru`** под *аккаунтом* пользователя **`user`** и попытайтесь получить с *ftp-сервера* файл **`pt3000-i6q4l2-mz.121-22.EA4.bin`**. Для получения справки по *ftp*-командам введите в строке приглашения знак вопроса и нажмите **Enter**.

8. Проанализируйте *TCP*-пакеты, проходящие по сети в процессе подключения к *ftp*-серверу, запроса справки, получения файла, а также других запросов, которые Вам приходится выполнять для полноценного взаимодействия с сервером.

9. Настройте почтовый сервер на хостинг-сервере **`www.corp.ru`**. Укажите используемое доменное имя (*Domain Name*) почтового сервера, не забудьте нажать кнопку *Set*. Обратите внимание, что введенное Вами доменное имя должно опознаваться доменной системой имен. Заведите несколько учетных записей пользователей, имена и пароли придумайте самостоятельно.

10. Сконфигурируйте клиенты *e-mail* на компьютерах **`192.168.5.a`**, **`192.168.5.b`** и **`192.168.5.c`** таким образом, чтобы они могли обращаться к разным учетным записям на сервере **`www.corp.ru`**. Отправьте несколько писем между ними. Обратите внимание, что новая почта автоматически в почтовых клиентах не появляется, для ее проверки необходимо нажать кнопку *Receive*.

11. Проанализируйте содержимое протокольных блоков данных с точки зрения модели *OSI* для следующих случаев:

- *DNS*-запрос;
- запрос браузером страницы с сайта;
- получение *ftp*-клиентом файла с *ftp*-сервера;
- отправка электронной почты;
- получение электронной почты по протоколу *POP3*.

Контрольные вопросы

1. Объясните принцип работы технологии *клиент–сервер*.
2. Что такое сервер? Что такое клиент?
3. Какие Вы можете назвать разновидности серверов?
4. Что такое протокол? Назовите наиболее широко распространенные протоколы.
5. Приведите характеристики протокола *TCP*.

6. Что такое протокол *UDP*? В каких случаях он используется и вместо какого протокола?

7. Что такое порт *TCP/IP*? Что такое сокет? За что отвечает порт источника и за что – порт назначения?

8. Назовите категории портов. Сколько портов может быть одновременно открыто на одном узле?

9. Что такое служба доменных имен?

10. Что представляет собой служба *WWW*?

11. Обязательно ли в начале имени сайта указывать префикс **`www`**, например: **`www.yandex.ru`**? С чем это связано?

12. Для чего нужна служба *FTP* и в чем ее отличия от *WWW*?

13. Какие протоколы отвечают за функционирование электронной почты?

14. Проведите аналогию между работой обычной бумажной почты и всеми тремя основными протоколами электронной почты.

15. Какие многоуровневые модели взаимодействия систем Вы можете назвать?

16. Подробно опишите процесс передачи веб-страницы от сервера к клиенту на каждом уровне модели *OSI*.